Лабораторная работа №3. Транспонирование матрицы и исследование локальности памяти

Марданов Самир, группа 11-401. Вариант C: тип double, $N_v = 4000$

1. Характеристики системы

Apple M1 Pro: 8 ядер (6 производительных + 2 энергоэффективных), кеш-линия 128 Б, страница 16 КБ. L1d: 128 КБ на P-ядро, 64 КБ на E-ядро; L2: 12 МБ общий на P-кластер, 4 МБ на E-кластер; отдельного L3 нет (System Level Cache 24 МБ); ОЗУ 16 ГБ LPDDR5. macOS 26.6, компилятор GCC 16.2.0 (Homebrew), `gcc transpose.c -O2 -fopenmp`. Время — `omp_get_wtime()`, медиана 5 запусков после одного прогревочного; все запуски прошли проверку (check=OK).

2. Результаты экспериментов

Эксперимент 1. Порядок обхода (1 поток)

N	rows, с	cols, с	BW rows, ГБ/с	BW cols, ГБ/с
256	0,000203	0,000048	5,17	21,85
512	0,000968	0,000246	4,33	17,05
1024	0,00487	0,00129	3,44	13,00
2048	0,0252	0,0138	2,66	4,86
4000	0,0572	0,1858	4,47	1,38

Эксперимент 2. Размер блока (1 поток, N = 4000)

b	Время, с	BW, ГБ/с
4	0,0242	10,58
8	0,0217	11,79
16	0,0166	15,39
32 = b*	0,0148	17,35
64	0,0168	15,23
128	0,0156	16,40

Эксперимент 3. Потоки (N = 4000, b = 32, T_1 = 0,0572 с)

Потоки	rows_omp, с	S rows	blocked_omp, с	S blocked
1	0,0587	0,98	0,0144	3,97
2	0,0301	1,90	0,00838	6,82
4	0,0162	3,54	0,00515	11,12
8	0,0109	5,23	0,00436	13,12
16	0,0111	5,16	0,00410	13,94

Эксперимент 4. False sharing (8 потоков, N = 4000, rows_omp)

Стратегия	Время, с	BW, ГБ/с
schedule(static)	0,0111	22,98
schedule(static, 1)	0,0191	13,37
schedule(static, 8)	0,0114	22,54

Доп. задание (in-place, N = 4000, b = 32): 1 поток — 0,00813 с; 8 потоков — 0,00230 с (blocked_omp, 8 потоков — 0,00436 с, т.е. in-place быстрее в 1,9 раза).

3. Вывод

Версия rows читает A последовательно, но пишет в B с шагом N·8 байт, cols — наоборот; в обоих случаях из каждой кеш-линии (128 Б) по «шаговой» матрице используется только 8 байт. Пока матрицы помещаются в кеш (дополнительно проверено при N = 250, 500, 1000), времена rows и cols совпадают в пределах 10 %. При N = 256…2048 шаг кратен 2 КБ, элементы одного столбца попадают в одни и те же наборы кеша и вытесняют друг друга (конфликты ассоциативности) — сильнее всего это бьёт по записи, поэтому rows медленнее cols в 2–4 раза. При N = 4000 (2 × 128 МБ, далеко за пределами кеша) картина обратная: cols медленнее в 3,2 раза, т.к. промах при чтении останавливает вычисления, а промахи записи скрываются буфером записи. Оптимум b* = 32: два блока 32 × 32 занимают 2 · 8 КБ = 16 КБ — малую часть L1d (128 КБ), а 32 строки блока лежат на 32 страницах и укладываются в TLB; при b ≤ 16 кеш-линия (16 double) используется не полностью, а при b ≥ 64 блоки начинают вытеснять друг друга из L1 и растут промахи TLB. Ускорение ограничено сильнее, чем для интеграла из ЛР2 (5,4 на 8 потоках), потому что транспонирование не вычисляет, а только перекладывает данные: собственное ускорение blocked_omp относительно blocked в 1 поток лишь 3,3, а BW насыщается на ~60 ГБ/с — все ядра делят одну подсистему памяти, и новые потоки не добавляют полосы. schedule(static, 1) медленнее static в 1,7 раза из-за false sharing: соседние строки A достаются разным потокам, и они пишут в соседние элементы B[j·N + i], B[j·N + i + 1] одной кеш-линии, которая постоянно инвалидируется между ядрами; при static каждый поток получает непрерывный диапазон из 500 строк, а при static, 8 — 64 байта, т.е. линию делят максимум два потока, и потери почти нет. In-place версия быстрее, так как работает с одним массивом и вдвое уменьшает объём трафика к памяти.